

# **15. Repaso análisis de texto.**

**Ciencia de datos para la toma de decisiones II**

**Jorge  
Juvenal  
Campos Ferreira**

✉ [juvenal.campos@tec.mx](mailto:juvenal.campos@tec.mx)

# ***Web scraping desde JS***

El web scraping es una técnica para **extraer datos de sitios web**.

Sin embargo, **los sitios web modernos a veces presentan desafíos únicos** debido a su arquitectura basada en **JavaScript** y renderizado dinámico.

Hay **técnicas para extraer contenido de sitios que utilizan JSON embebido**, un patrón común en aplicaciones web contemporáneas.

# Web scraping desde JS

En el modelo tradicional, el servidor genera HTML completo con todo el contenido:

```
<article>
  <p>Párrafo 1 del artículo</p>
  <p>Párrafo 2 del artículo</p>
  <p>Párrafo 3 del artículo</p>
</article>
```

Este enfoque es directo para web scraping: las herramientas como `rvest` pueden leer directamente los elementos `<p>` y extraer su contenido.

# Web scraping desde JS

## Renderizado del Lado del Cliente (Client-Side Rendering - CSR)

Los sitios modernos, especialmente aquellos contruidos con frameworks como React, Vue o Angular, funcionan de manera diferente:

1. El servidor envía un HTML mínimo con un contenedor vacío
2. El contenido real está en formato JSON dentro de un tag `<script>`
3. JavaScript del navegador lee ese JSON y genera dinámicamente el HTML
4. El usuario ve el contenido renderizado, pero el HTML original estaba vacío

Estructura típica:

```
<script>
  window.__INITIAL_STATE__ = {
    "type": "storyline_paragraph",
    "fields": {
      "paragraph": {"value": "Párrafo 1 del artículo"}
    }
  }
</script>

<div id="root"></div>
```

# Web scraping desde JS

## Caso de Estudio: El Sol de Toluca

El sitio web de El Sol de Toluca (oem.com.mx) utiliza esta arquitectura híbrida. Los artículos se estructuran en JSON con el siguiente formato:

```
{  
  "id": "af7f6d72-48c9-4184-ba5f-469a81d8a017",  
  "type": "storyline_paragraph",  
  "fields": {  
    "paragraph": {  
      "value": "En el Estado de México continúan detectándose...",  
      "annotations": [...]  
    }  
  }  
}
```



# Web scraping desde JS

## Función de Extracción de Párrafos

La función principal utiliza expresiones regulares para identificar y extraer los párrafos del JSON embebido:

```
extraer_parrafos <- function(json_text) {  
  # Patrón regex para encontrar párrafos en el JSON  
  # Busca: "type": "storyline_paragraph" ... "paragraph": {"value": "TEXT0"}  
  patron <- '"type"\\s*:\\s*"storyline_paragraph"[^}]+?"paragraph"\\s*:\\s*\\{\\s*'  
  
  # Extraer todas las coincidencias  
  matches <- str_match_all(json_text, patron)[[1]]  
  
  if (nrow(matches) > 0) {  
    # La segunda columna contiene el texto capturado  
    parrafos <- matches[, 2]  
  
    # Limpiar caracteres escapados comunes en JSON  
    parrafos <- gsub("\\\\n", "\\n", parrafos) # Saltos de línea  
    parrafos <- gsub("\\\\\"", "\"", parrafos) # Comillas  
  
    return(parrafos)  
  }  
  
  return(character(0))  
}
```

# Web scraping desde JS

## Explicación del patrón regex:

- `"type"\\s*:\\s*"storyline_paragraph"` - Identifica el tipo de contenido
- `[^}]+?` - Captura cualquier contenido hasta la siguiente llave
- `"paragraph"\\s*:\\s*\\{\\s*"value"\\s*:\\s*` - Navega hasta el valor del párrafo
- `( [^"]+ )` - **Grupo de captura:** extrae el texto entre comillas

# Ejercicio 01

**Revisar el código de web scraping del Sol de Toluca.**

## Modelo de clasificación (cont.)

Para trabajar con texto, un modelo de clasificación supervisada requiere de dos elementos básicos:

Un vector **y** de etiquetas.

Una matriz **X** de covariantes/datos dependientes. Esta generalmente será la matriz documento-término.

# Función *predict*

La función *predict* es una función genérica para realizar **predicciones** a partir de los resultados de diversos **modelos** estadísticos.

En este contexto:

**predict(object, ...)**

Donde **object** es el **modelo** que predice y en los “...” le pasamos la **matriz *dtm*** con los datos de frecuencia/tf-idf a los que queremos hacer la predicción.

**Por ejemplo**, si le pasamos a la función `predict()` el modelo entrenado y la matriz *dtm* de la base de validación, esta nos va a devolver una serie de etiquetas con los valores predichos para el modelo.

## Ejercicio 02

1. Revisar el código **sesion\_14\_clase.R**
2. Revisar la parte de la generación del modelo a través de la funcion `randomForest()`
3. Revisar el uso de la función **predict()**. ¿Cuál es la precisión con la base de entrenamiento? ¿Y con la base de validación/prueba/test?

## Ejercicio 03

Elabore un modelo similar al de la sesión 14, pero ahora para los comentarios del siguiente video: <https://www.youtube.com/watch?v=hcatn-d-TSo>.

### Pasos a seguir:

1. Cargue los datos del archivo de excel “comentarios\_little\_america.xlsx”
2. Revise el proceso de etiquetado del script “02\_Codigo/etiquetado.R”. Recuerde que para ejecutar este código deberá tener Ollama instalado, así como los dos modelos mencionados en el código.
3. Limpie acentos, defina *bigramas* o *trigramas* de interés, considere la conveniencia de lematizar o podar palabras y elimine stopwords
4. Tokenice y obtenga la frecuencia de sus tokens para cada documento.
5. Genere su matriz ***dtm*** con la función **cast\_dtm**.

## Ejercicio 03

6. A partir de la matriz **dtm** y de sus etiquetas “**y**” obtenidas por Ollama, corra un modelo de clasificación (randomForest). Verifique que tanto **X** como **Y** **tengan las mismas dimensiones**.
7. Obtenga la precisión del modelo para su entrenamiento y para su validación.
8. Pruebe el modelo con nuevos comentarios. Para ello, copie el código silenciado del script anterior.